



Quantization

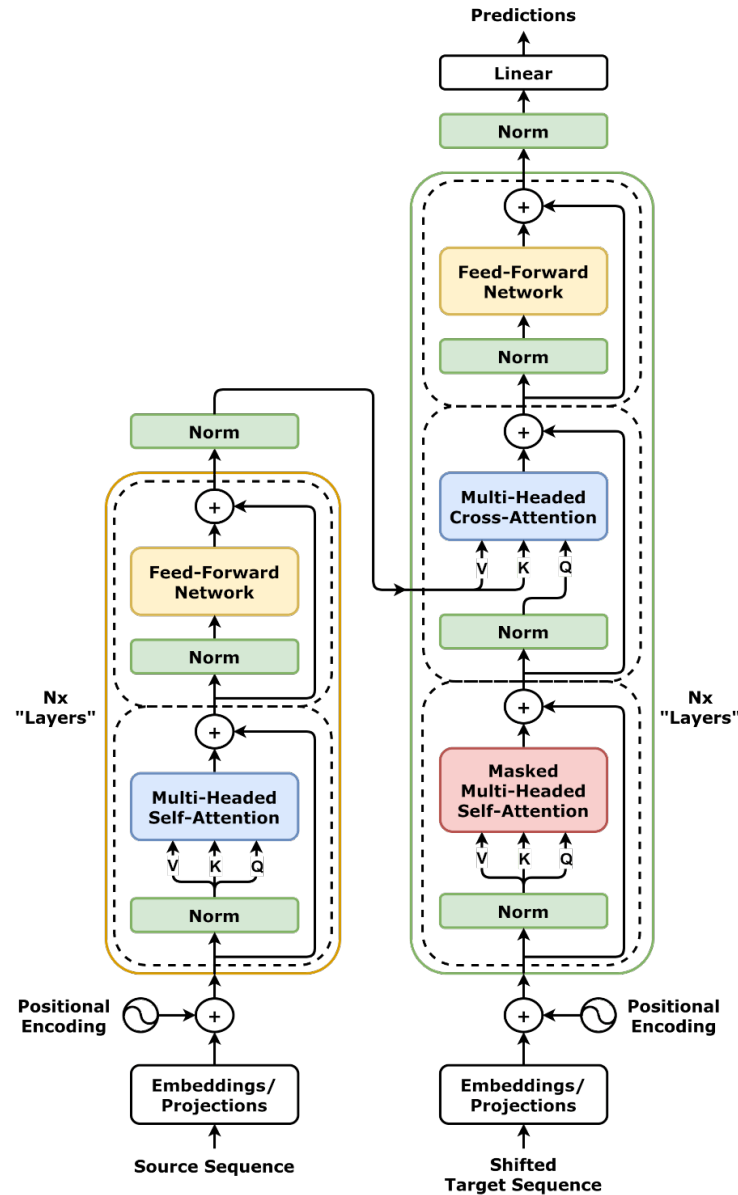
by Daniel Voigt Godoy
Data Science Retreat – May 2025



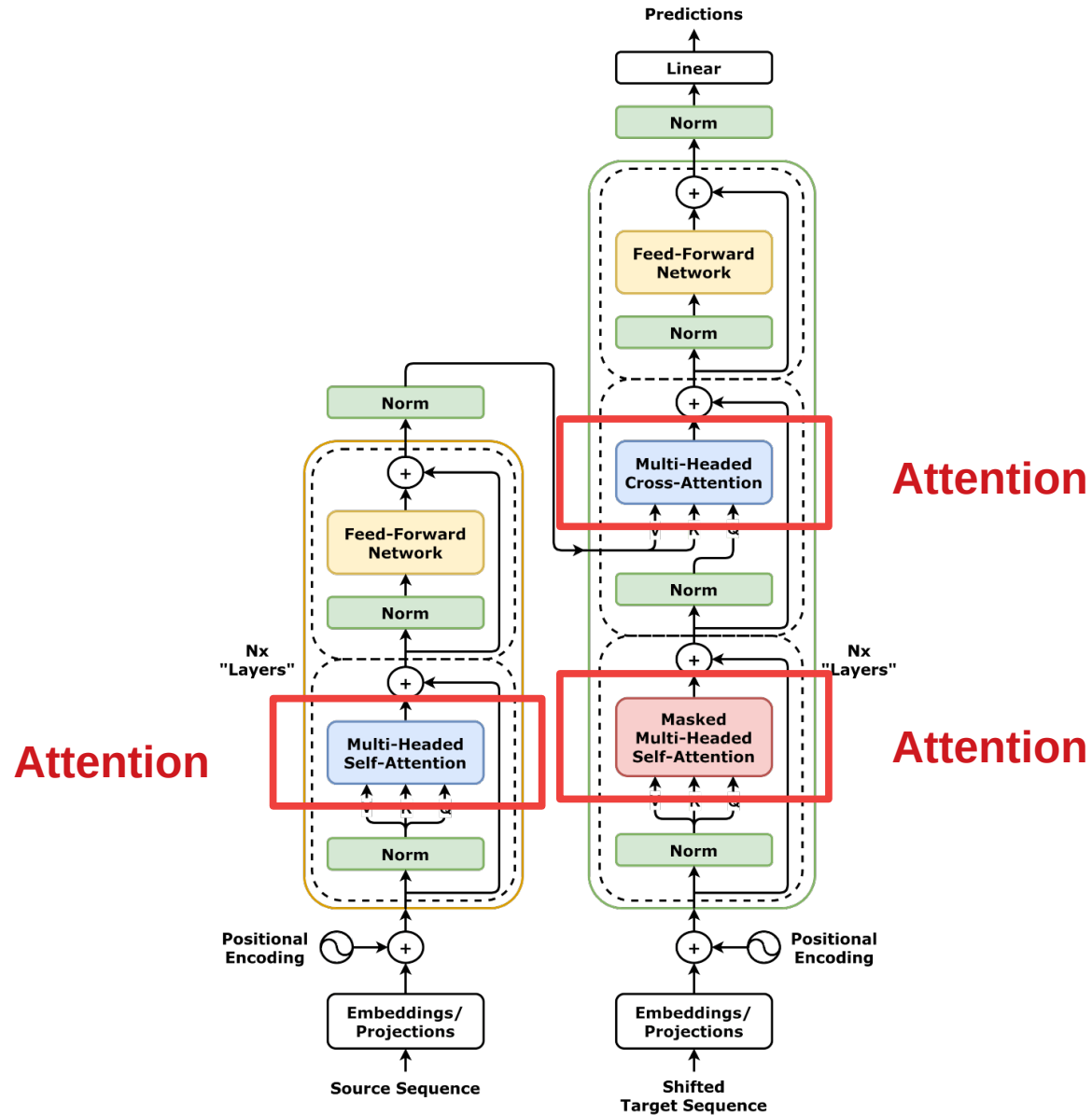
Why Quantize?

- To lower memory usage
 - Transformers are HUGE
 - Models do not fit into GPU RAM
 - FP32 is wasteful
- For faster inference
 - Loads weights faster
 - Integer arithmetics runs faster

Attention is All You Need



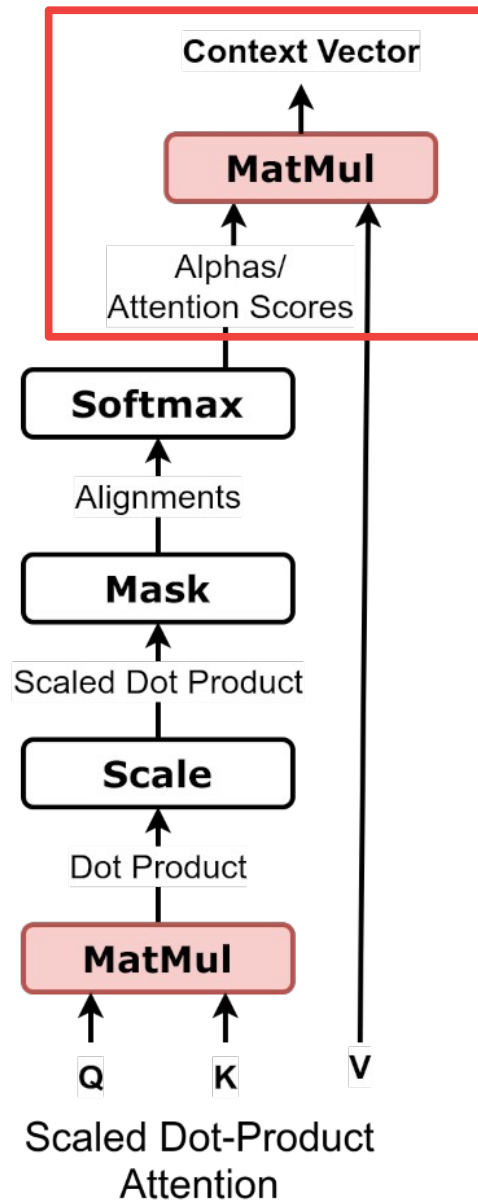
Attention is All You Need



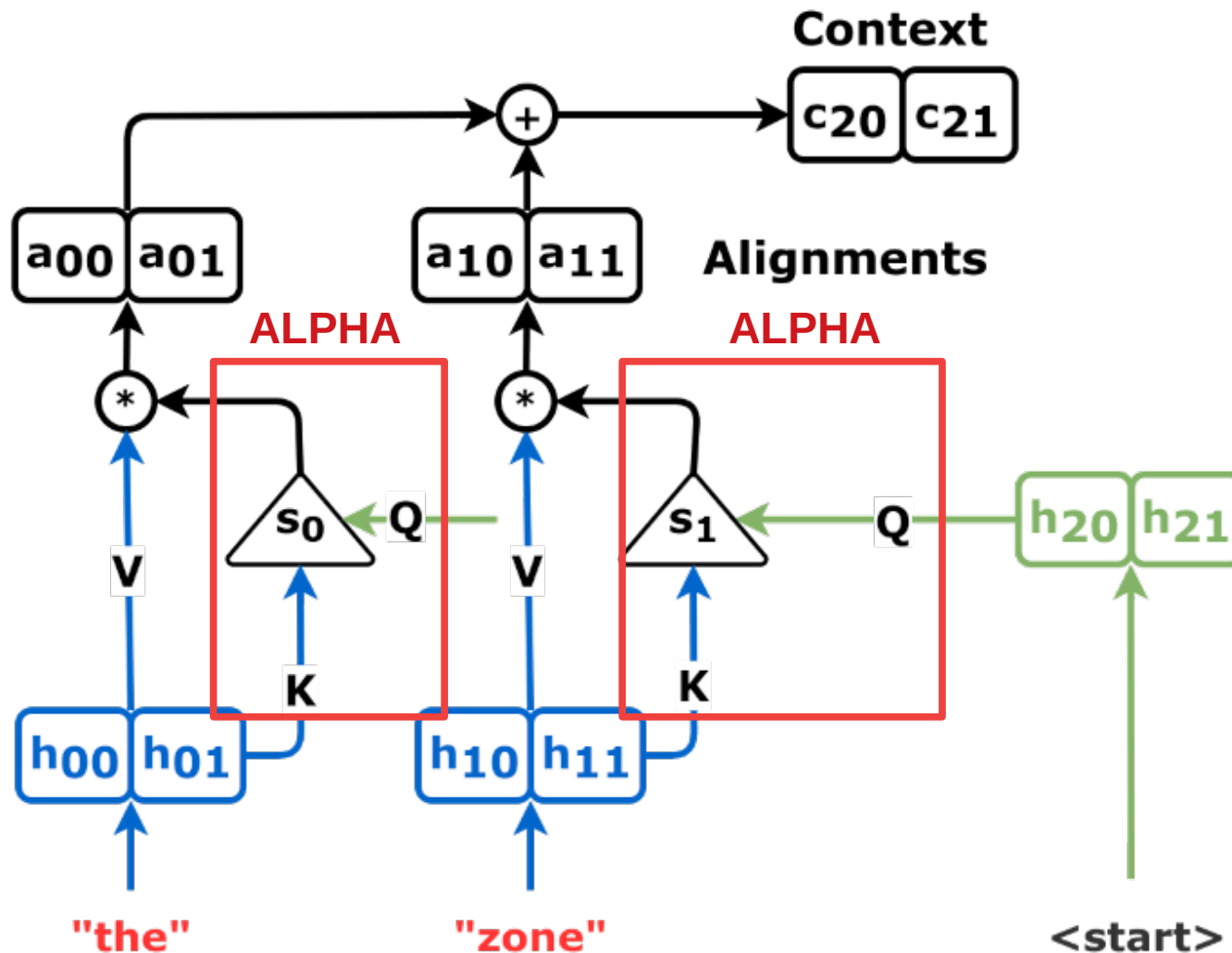
Attention is All You Need

	the	European	economic	zone
la	0.80	0.00	0.00	0.20
zone	0.00	0.00	0.00	1.00
économique	0.00	0.00	1.00	0.00
européenne	0.00	0.80	0.00	0.20

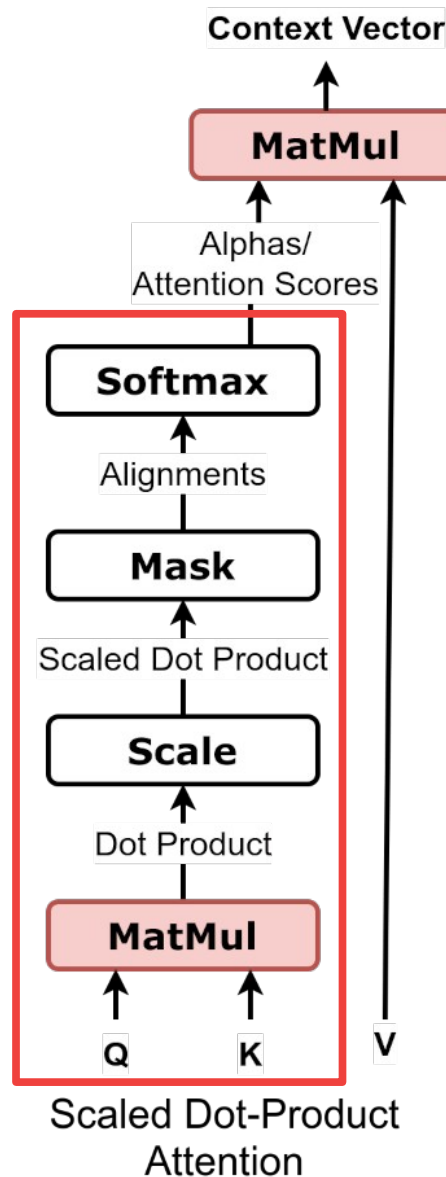
Attention is All You Need



Attention is All You Need



Attention is All You Need



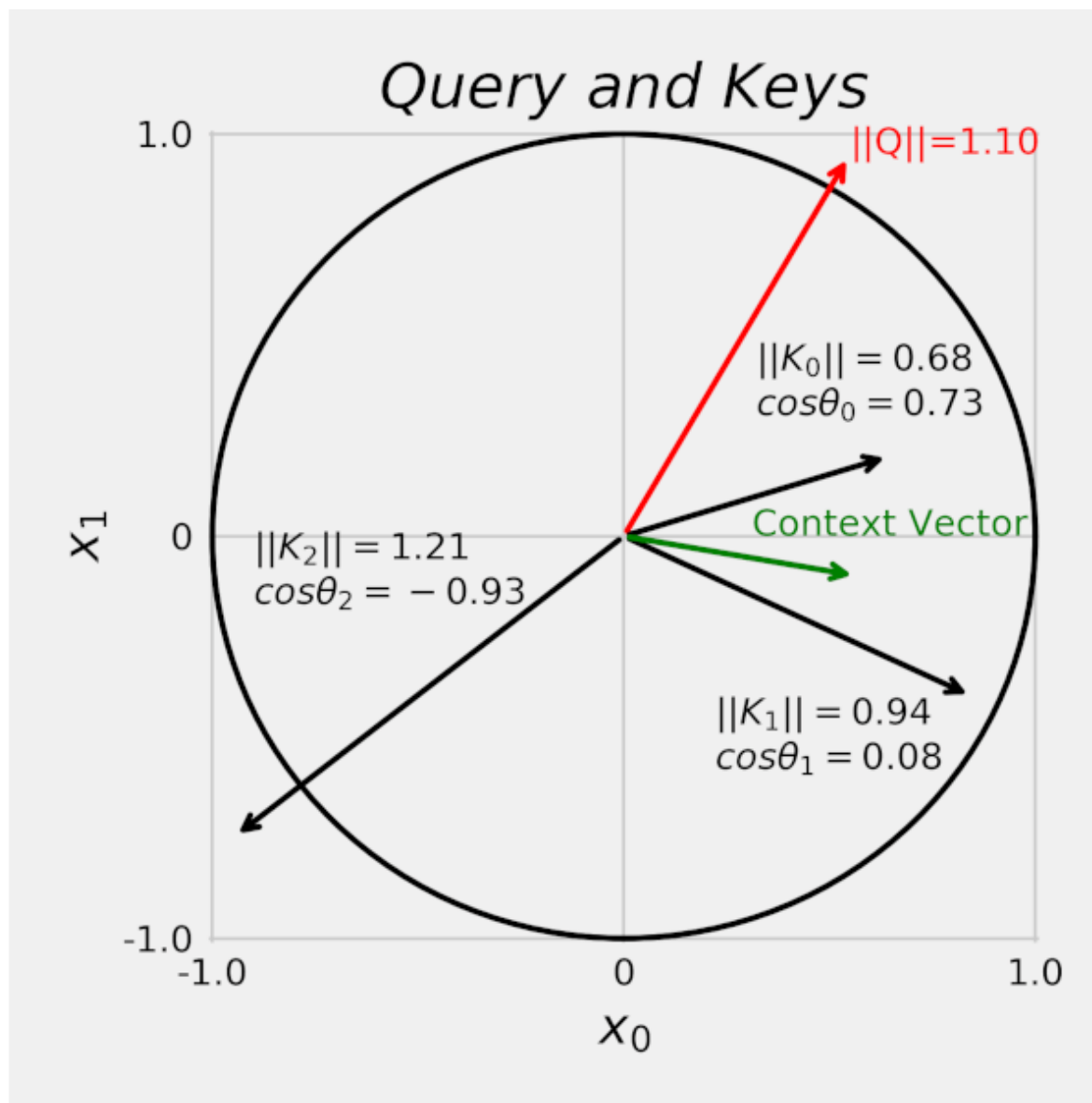
Attention is All You Need

$$\text{scaled dot product} = \frac{Q \cdot K}{\sqrt{d_k}}$$

```
n_dims = 10  
vector1 = torch.randn(10000, 1, n_dims)  
vector2 = torch.randn(10000, 1, n_dims).permute(0, 2, 1)  
torch.bmm(vector1, vector2).squeeze().var()
```

$$\cos \theta \ ||Q|| \ ||K|| = Q \cdot K$$

Attention is All You Need



Attention is All You Need

$$\alpha_{00}, \alpha_{01} = \text{softmax}\left(\frac{Q_0 \cdot K_0}{\sqrt{2}}, \frac{Q_0 \cdot K_1}{\sqrt{2}}\right)$$

$$\text{context vector}_0 = \alpha_{00}V_0 + \alpha_{01}V_1$$

$$\alpha_{10}, \alpha_{11} = \text{softmax}\left(\frac{Q_1 \cdot K_0}{\sqrt{2}}, \frac{Q_1 \cdot K_1}{\sqrt{2}}\right)$$

$$\text{context vector}_1 = \alpha_{10}V_0 + \alpha_{11}V_1$$

Attention is All You Need

```
1 class Attention(nn.Module):
2     def __init__(self, hidden_dim, input_dim=None,
3                 proj_values=False):
4         super().__init__()
5         self.d_k = hidden_dim
6         self.input_dim = hidden_dim if input_dim is None \
7             else input_dim
8         self.proj_values = proj_values
9         # Affine transformations for Q, K, and V
10        self.linear_query = nn.Linear(self.input_dim, hidden_dim)
11        self.linear_key = nn.Linear(self.input_dim, hidden_dim)
12        self.linear_value = nn.Linear(self.input_dim, hidden_dim)
13        self.alphas = None
14
15    def init_keys(self, keys):
16        self.keys = keys
17        self.proj_keys = self.linear_key(self.keys)
18        self.values = self.linear_value(self.keys) \
19            if self.proj_values else self.keys
```

**PROJECTION
LAYERS**

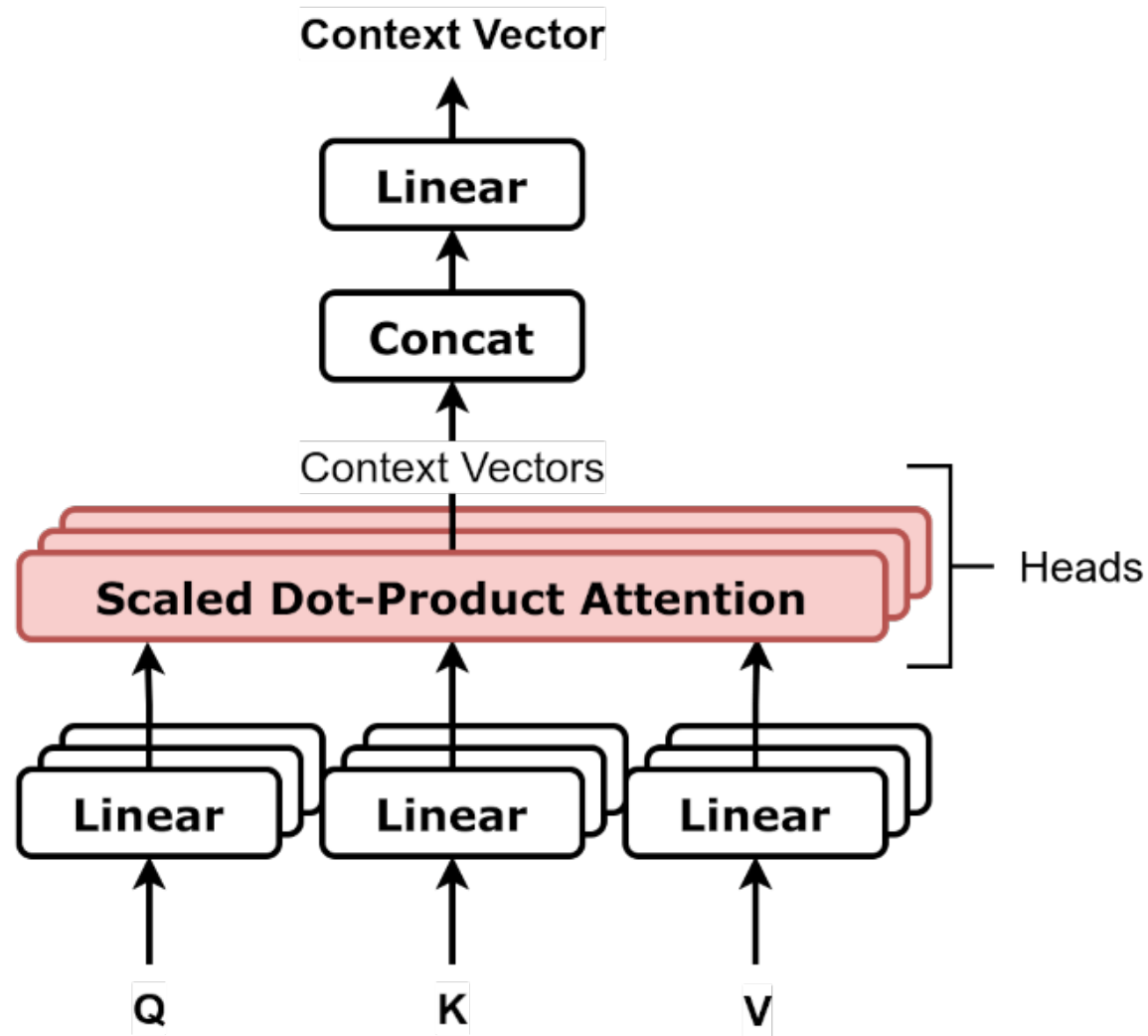
**Projections of Ks and Vs
or simply Ks and Vs**

Attention is All You Need

```
20
21 def score_function(self, query):
22     proj_query = self.linear_query(query)
23     # scaled dot product
24     # N, 1, H x N, H, L -> N, 1, L
25     dot_products = torch.bmm(proj_query,
26                               self.proj_keys.permute(0, 2, 1))
27     scores = dot_products / np.sqrt(self.d_k)
28     return scores
29
30 def forward(self, query, mask=None):
31     # Query is batch-first N, 1, H
32     scores = self.score_function(query) # N, 1, L ①
33     if mask is not None:
34         scores = scores.masked_fill(mask == 0, -1e9)
35     alphas = F.softmax(scores, dim=-1) # N, 1, L ②
36     self.alphas = alphas.detach()
37
38     # N, 1, L x N, L, H -> N, 1, H
39     context = torch.bmm(alphas, self.values) ③
40     return context
```

Projections of Qs

Multi-Head Attention (MHA)

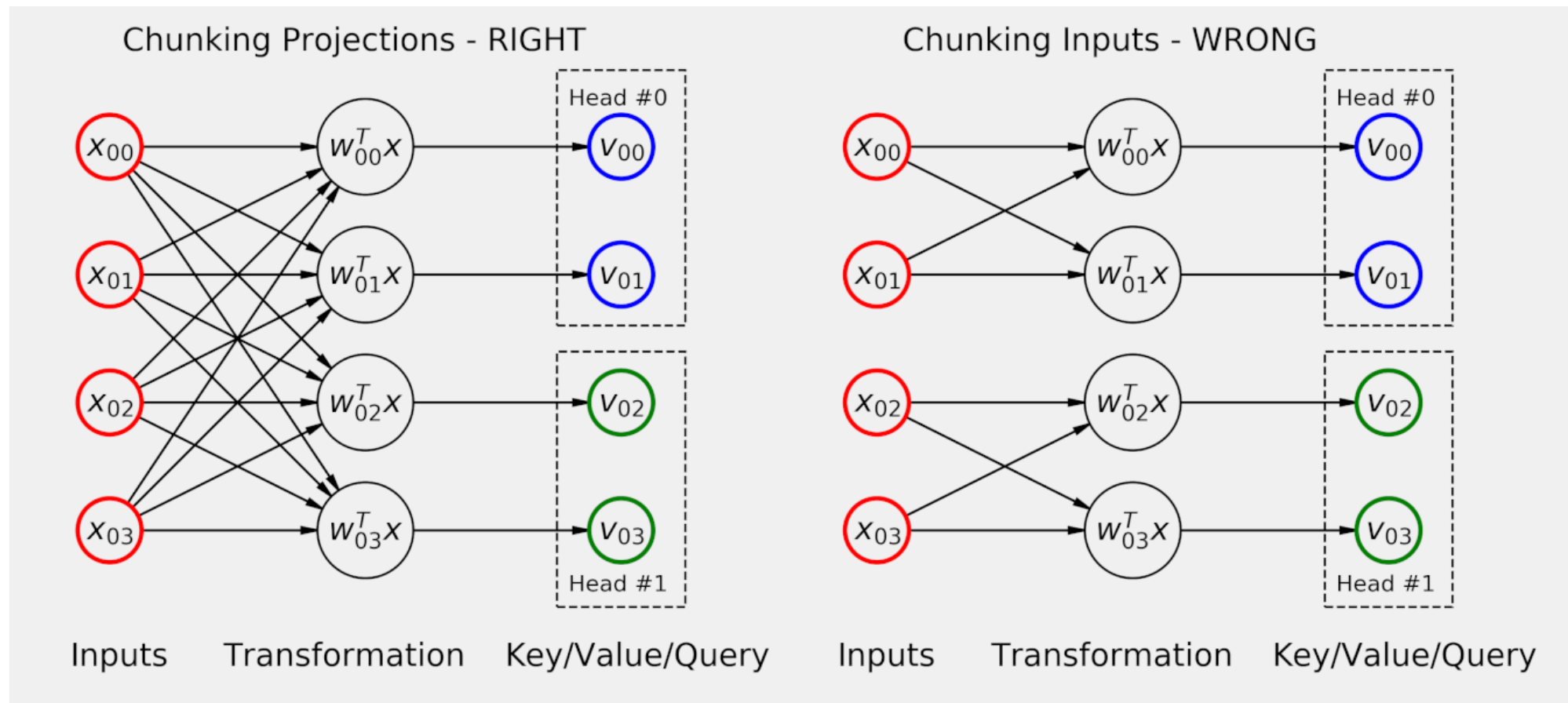


Multi-Head Attention (MHA)

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, n_heads, d_model,
3                   input_dim=None, proj_values=True):
4         super().__init__()
5         self.linear_out = nn.Linear(n_heads * d_model, d_model)
6         self.attn_heads = nn.ModuleList(
7             [Attention(d_model,
8                       input_dim=input_dim,
9                       proj_values=proj_values)
10              for _ in range(n_heads)]
11         )
12
13     def init_keys(self, key):
14         for attn in self.attn_heads:
15             attn.init_keys(key)
16
17     @property
18     def alphas(self):
19         # Shape: n_heads, N, 1, L (source)
20         return torch.stack(
21             [attn.alphas for attn in self.attn_heads], dim=0
22         )
23
24     def output_function(self, contexts):
25         # N, 1, n_heads * D
26         concatenated = torch.cat(contexts, axis=-1)
27         # Linear transf. to go back to original dimension
28         out = self.linear_out(concatenated) # N, 1, D
29         return out
30
31     def forward(self, query, mask=None):
32         contexts = [attn(query, mask=mask)
33                     for attn in self.attn_heads]
34         out = self.output_function(contexts)
35         return out
```

Multiple heads!

Multi-Head Attention (MHA)



Multi-Head Attention (MHA)

```
1 class MultiHeadedAttention(nn.Module):
2     def __init__(self, n_heads, d_model, dropout=0.1):
3         super(MultiHeadedAttention, self).__init__()
4         self.n_heads = n_heads
5         self.d_model = d_model
6         self.d_k = int(d_model / n_heads) ①
7         self.linear_query = nn.Linear(d_model, d_model)
8         self.linear_key = nn.Linear(d_model, d_model)
9         self.linear_value = nn.Linear(d_model, d_model)
10        self.linear_out = nn.Linear(d_model, d_model)
11        self.dropout = nn.Dropout(p=dropout) ④
12        self.alphas = None
13
14    def make_chunks(self, x): ①
15        batch_size, seq_len = x.size(0), x.size(1)
16        # N, L, D -> N, L, n_heads * d_k
17        x = x.view(batch_size, seq_len, self.n_heads, self.d_k)
18        # N, n_heads, L, d_k
19        x = x.transpose(1, 2)
20        return x
21
22    def init_keys(self, key):
23        # N, n_heads, L, d_k
24        self.proj_key = self.make_chunks(self.linear_key(key)) ①
25        self.proj_value = \
26            self.make_chunks(self.linear_value(key)) ①
27
```

PROJECTION
LAYERS

CHUNKING

CHUNKING

Multi-Head Attention (MHA)

```
27
28     def score_function(self, query):
29         # Scaled dot product
30         proj_query = self.make_chunks(self.linear_query(query))①
31         # N, n_heads, L, d_k x N, n_heads, d_k, L ->
32         # N, n_heads, L, L
33         dot_products = torch.matmul(
34             proj_query, self.proj_key.transpose(-2, -1)
35         )
36         scores = dot_products / np.sqrt(self.d_k)
37         return scores
38
39     def attn(self, query, mask=None):
40         # Query is batch-first: N, L, D
41         # Score function will generate scores for each head
42         scores = self.score_function(query) # N, n_heads, L, L
43         if mask is not None:
44             scores = scores.masked_fill(mask == 0, -1e9)
45         alphas = F.softmax(scores, dim=-1) # N, n_heads, L, L
46
47         alphas = self.dropout(alphas)
48         self.alphas = alphas.detach()
49
50         # N, n_heads, L, L x N, n_heads, L, d_k ->
51         # N, n_heads, L, d_k
52         context = torch.matmul(alphas, self.proj_value)
53         return context
54
```

CHUNKING

Multi-Head Attention (MHA)

```
55     def output_function(self, contexts):
56         # N, L, D
57         out = self.linear_out(contexts) # N, L, D
58         return out
59
60     def forward(self, query, mask=None):
61         if mask is not None:
62             # N, 1, L, L - every head uses the same mask
63             mask = mask.unsqueeze(1)
64
65         # N, n_heads, L, d_k
66         context = self.attn(query, mask=mask)
67         # N, L, n_heads, d_k
68         context = context.transpose(1, 2).contiguous() ⑤
69         # N, L, n_heads * d_k = N, L, d_model
70         context = context.view(query.size(0), -1, self.d_model) ⑤
71         # N, L, d_model
72         out = self.output_function(context)
73         return out
```

CONCAT
CHUNKS

Multi-Head Attention (MHA)

HEAD #0

FOUR
PROJECTIONS
FOR Q, K, V

q							
0.23	0.03	0.1	0.3	0.87	0.84	0.3	0.3
0.27	0.61	0.7	0.02	0.83	0.94	0.12	0.21
0.79	0.23	0.03	0.28	0.02	0.47	0.97	0.61
0.11	0.1	0.3	1.0	0.08	0.88	0.83	0.69
0.07	0.01	0.16	0.05	0.51	0.54	0.23	0.47

q/0		q/1		q/2		q/3	
0.23	0.03	0.1	0.3	0.87	0.84	0.3	0.3
0.27	0.61	0.7	0.02	0.83	0.94	0.12	0.21
0.79	0.23	0.03	0.28	0.02	0.47	0.97	0.61
0.11	0.1	0.3	1.0	0.08	0.88	0.83	0.69
0.07	0.01	0.16	0.05	0.51	0.54	0.23	0.47

k							
0.08	0.41	0.36	0.1	0.15	0.03	0.95	0.16
0.7	0.77	0.57	0.9	0.65	0.36	0.58	0.32
0.77	0.29	0.42	0.58	0.16	0.49	0.17	0.73
0.94	0.36	0.16	0.03	0.31	0.67	0.81	0.94
0.76	1.0	0.45	0.94	0.6	0.49	0.68	0.54

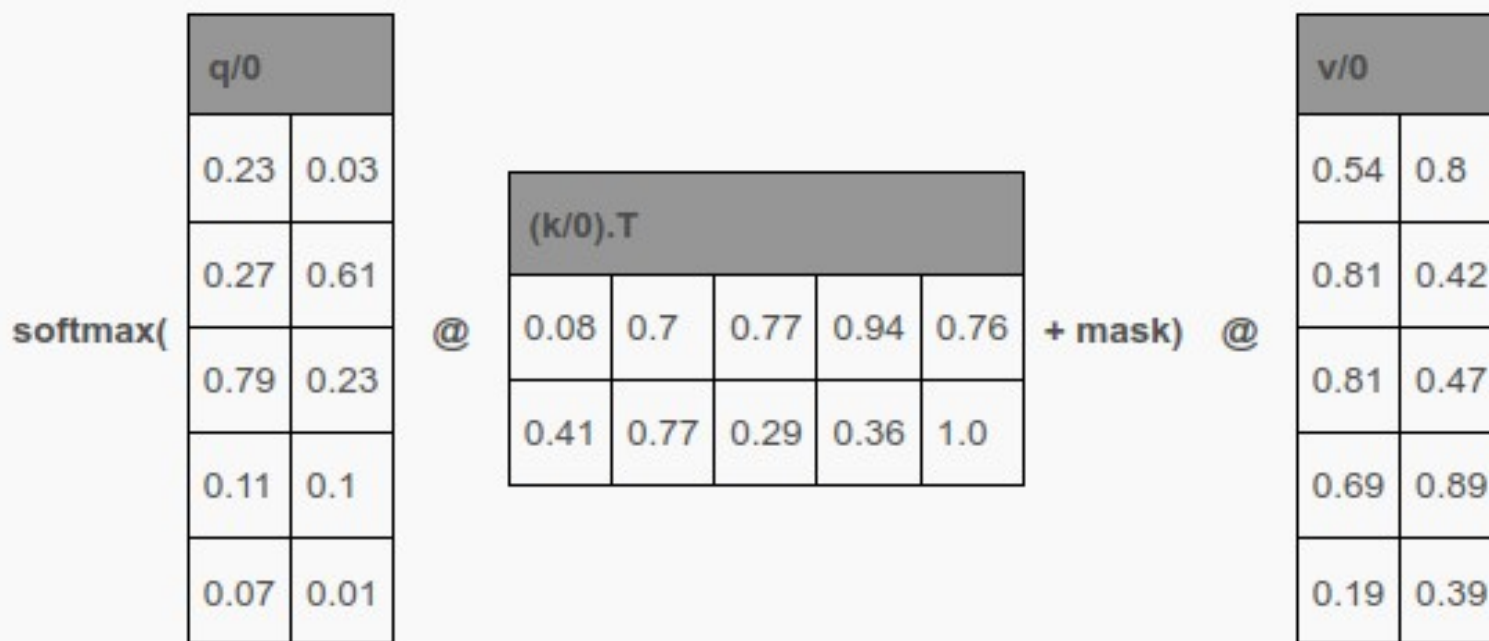
k/0		k/1		k/2		k/3	
0.08	0.41	0.36	0.1	0.15	0.03	0.95	0.16
0.7	0.77	0.57	0.9	0.65	0.36	0.58	0.32
0.77	0.29	0.42	0.58	0.16	0.49	0.17	0.73
0.94	0.36	0.16	0.03	0.31	0.67	0.81	0.94
0.76	1.0	0.45	0.94	0.6	0.49	0.68	0.54

v							
0.54	0.8	0.73	0.35	0.97	0.05	0.07	0.45
0.81	0.42	0.13	0.33	0.6	0.75	0.41	0.36
0.81	0.47	0.2	0.05	0.63	0.75	0.58	0.66
0.69	0.89	0.09	0.49	0.49	0.63	0.91	0.88
0.19	0.39	0.22	0.36	1.0	0.17	0.66	0.02

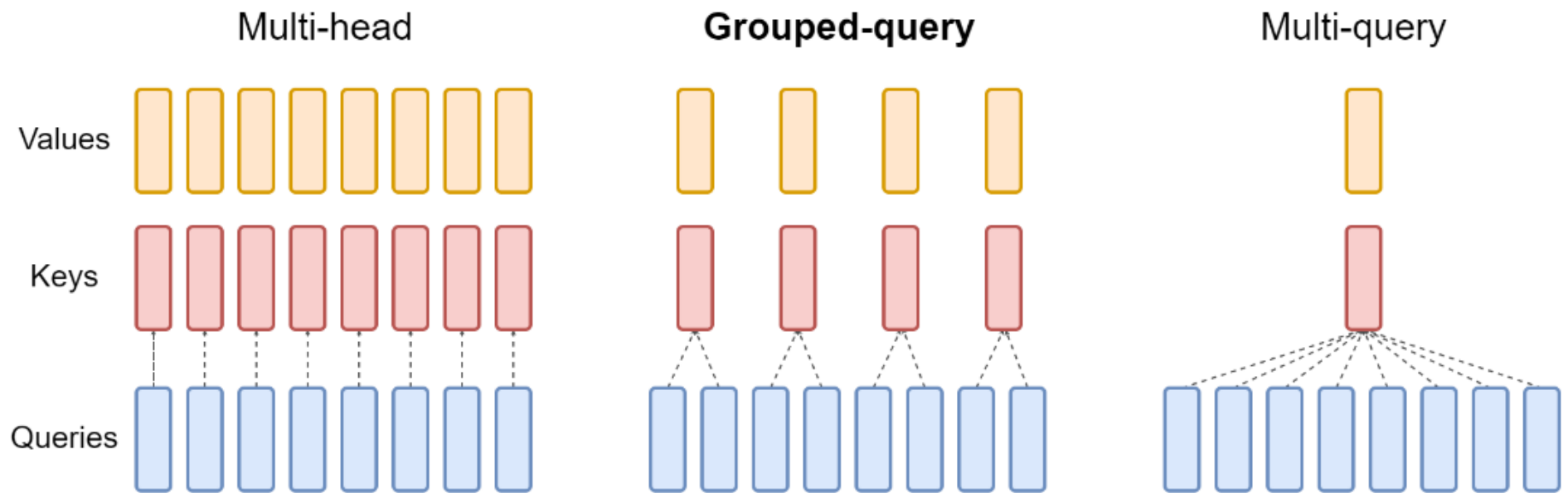
v/0		v/1		v/2		v/3	
0.54	0.8	0.73	0.35	0.97	0.05	0.07	0.45
0.81	0.42	0.13	0.33	0.6	0.75	0.41	0.36
0.81	0.47	0.2	0.05	0.63	0.75	0.58	0.66
0.69	0.89	0.09	0.49	0.49	0.63	0.91	0.88
0.19	0.39	0.22	0.36	1.0	0.17	0.66	0.02

Source: <https://vgel.me/posts/faster-inference>

Multi-Head Attention (MHA)



MHA, GQA, MQA



Multi-Query Attention(MQA)

FOUR
PROJEC-
TIONS
FOR Q

q							
0.23	0.03	0.1	0.3	0.87	0.84	0.3	0.3
0.27	0.61	0.7	0.02	0.83	0.94	0.12	0.21
0.79	0.23	0.03	0.28	0.02	0.47	0.97	0.61
0.11	0.1	0.3	1.0	0.08	0.88	0.83	0.69
0.07	0.01	0.16	0.05	0.51	0.54	0.23	0.47

→

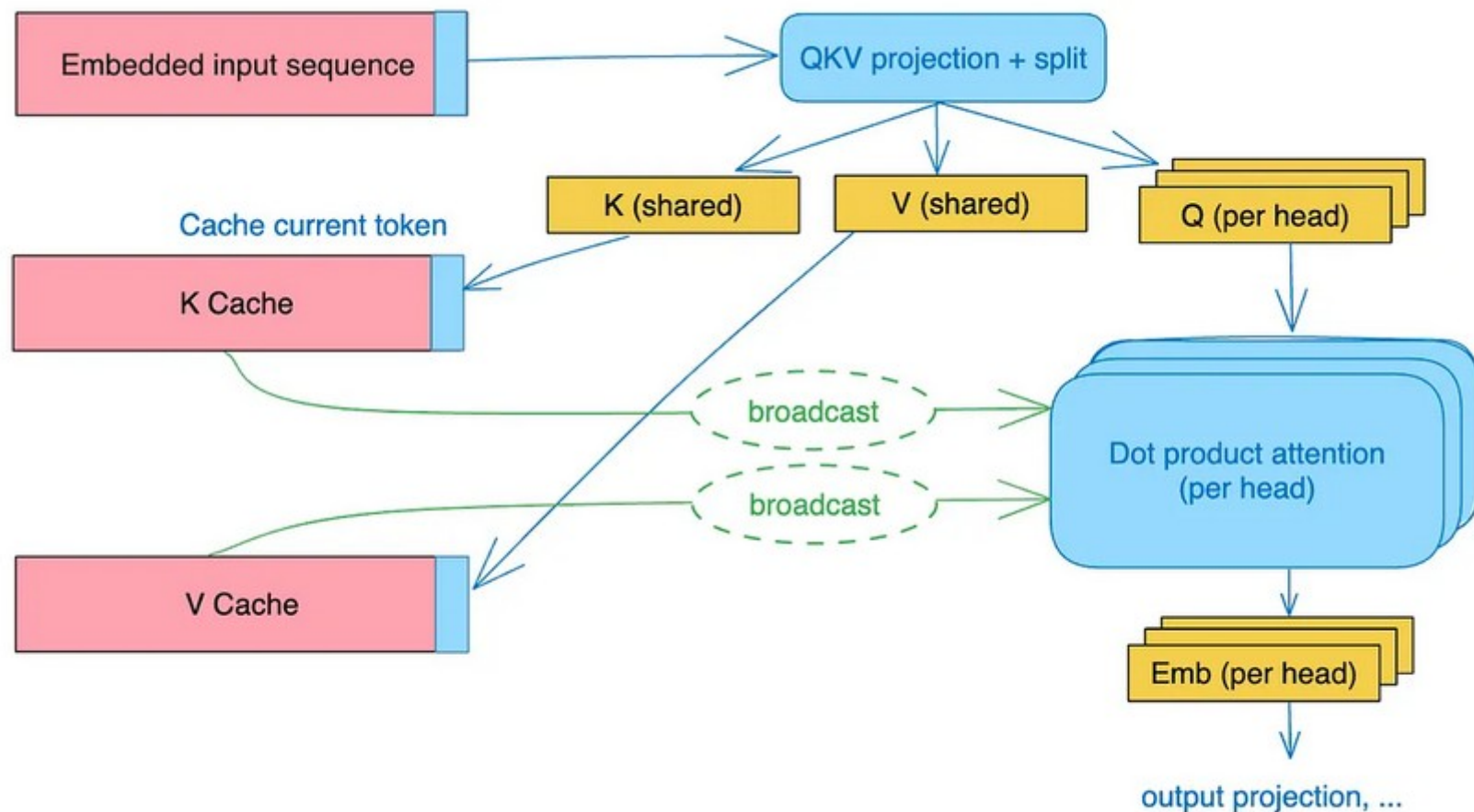
q/0		q/1		q/2		q/3	
0.23	0.03	0.1	0.3	0.87	0.84	0.3	0.3
0.27	0.61	0.7	0.02	0.83	0.94	0.12	0.21
0.79	0.23	0.03	0.28	0.02	0.47	0.97	0.61
0.11	0.1	0.3	1.0	0.08	0.88	0.83	0.69
0.07	0.01	0.16	0.05	0.51	0.54	0.23	0.47

SINGLE PROJECTIONS
FOR K AND V
(SHARED WITH EACH
PROJECTION OF Q)

k	
0.08	0.41
0.7	0.77
0.77	0.29
0.94	0.36
0.76	1.0

v	
0.54	0.8
0.81	0.42
0.81	0.47
0.69	0.89
0.19	0.39

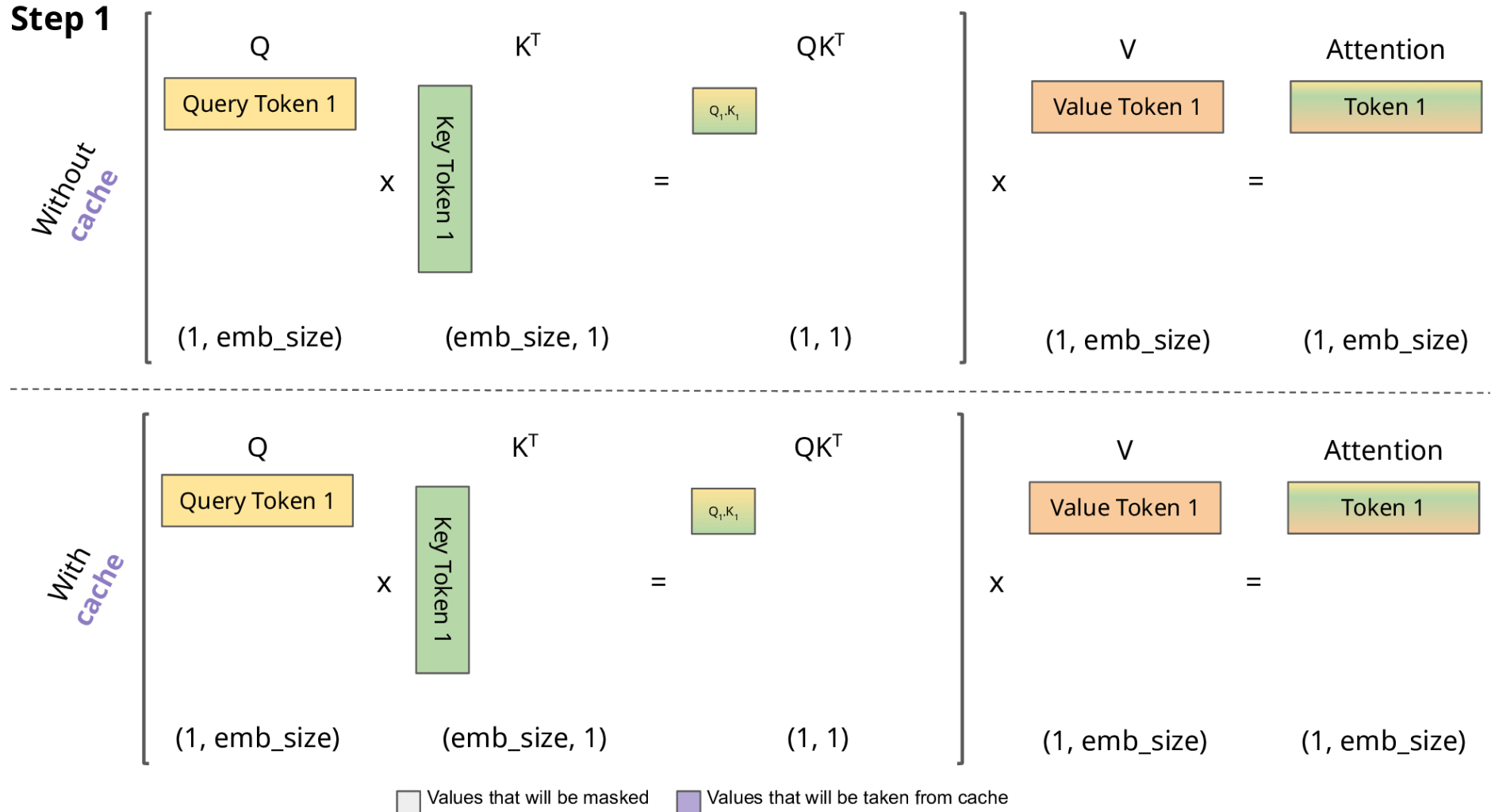
Multi-Query Attention(MQA)



Multi-Query Attention

KV Caching

Step 1





Hands-On

Notebook 1 - MHA vs MQA

Sparse Attention

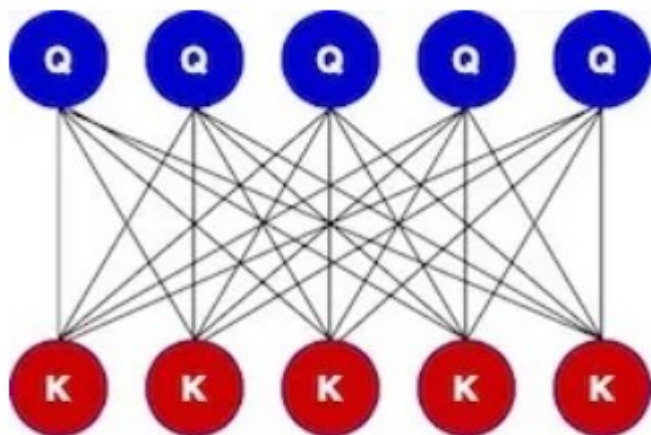


Figure 1. Attention Layer Representation

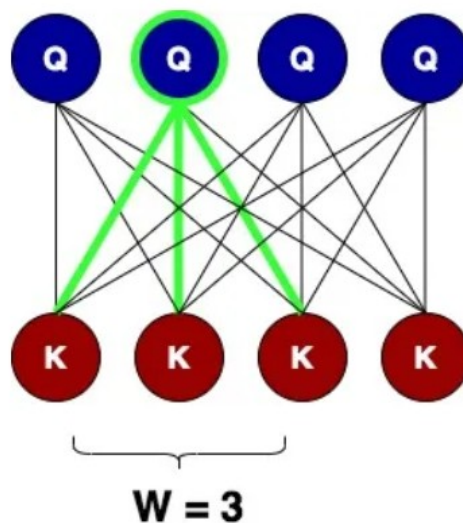


Figure 4. Sliding Attention Window of Size 3.

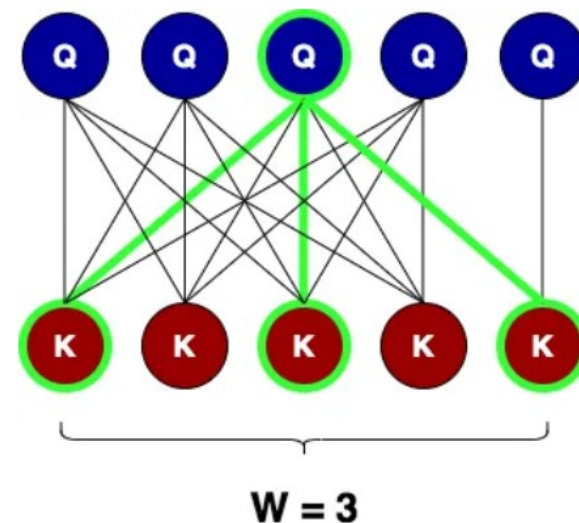
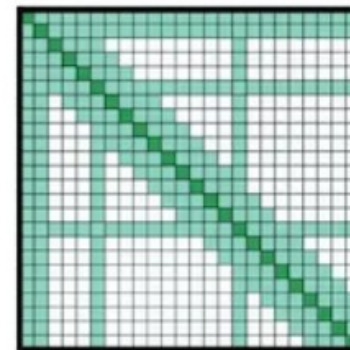
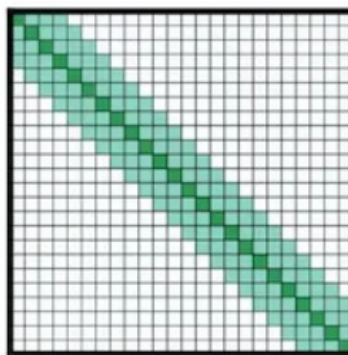
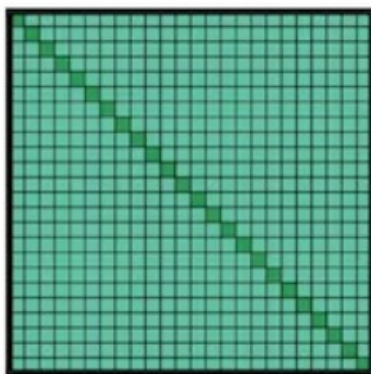
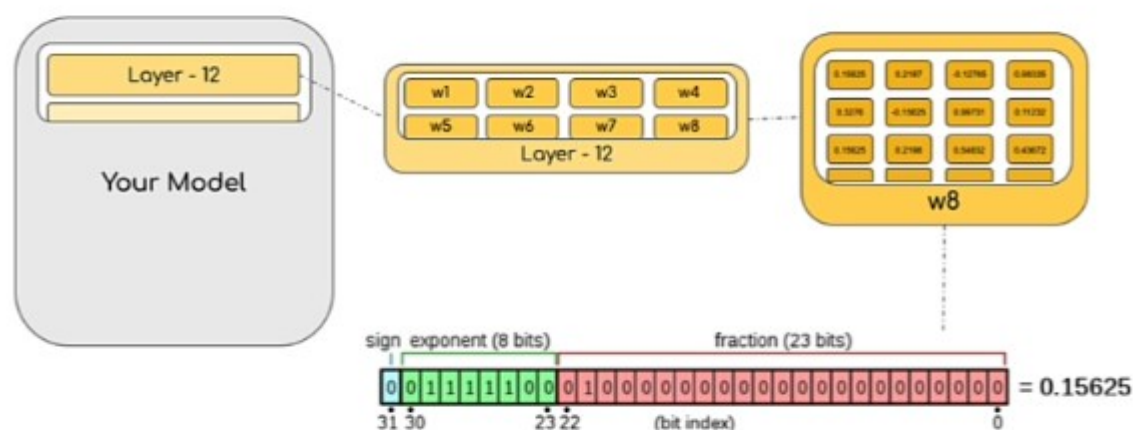


Figure 7. Dilated Sliding Window Attention Layer



Quantization

What is a model parameter?

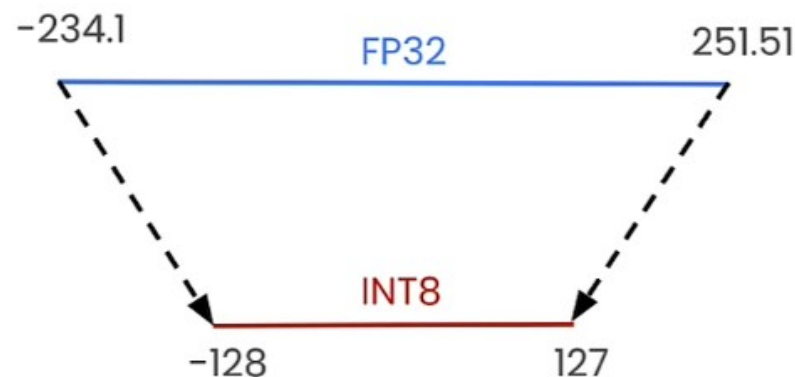
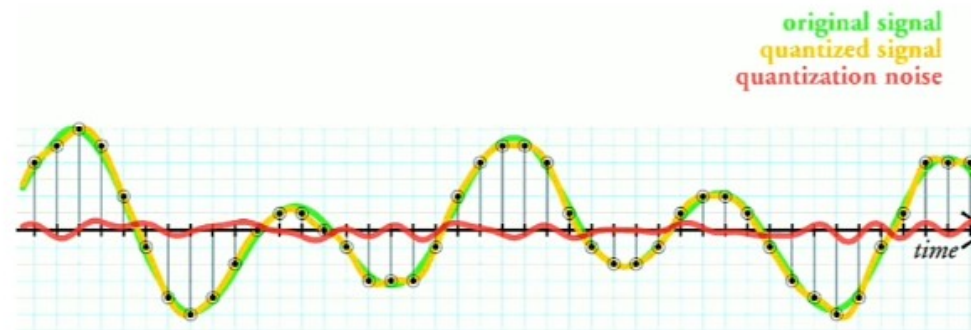


It is possible to inspect each parameter's data type!

Quantization

Quantization – Concept

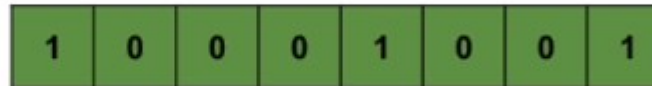
Quantization refers to the process of mapping a large set to a smaller set of values.



Quantization

Data representation in ML Dtypes

- Integer (int8)



- Floating Point (FP32, F16, BF16)

- **Sign**: 1 bit
- **Exponent** (range): 8 bit
- **Fraction** (precision): 23 bit
- **Total**: 32 bit

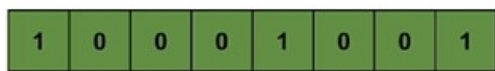


Quantization

Integer

- Unsigned integer:
 - Range for n-bits: $[0, 2^n - 1]$

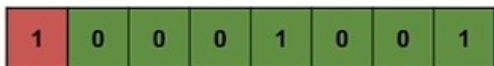
Example with 8-bit (torch.uint8): $[0, 255]$



$$\begin{array}{cccccccc} 2^7 & + & 0 & + & 0 & + & 0 & + & 0 & + & 2^3 & + & 0 & + & 0 & + & 2^0 & = & 137 \\ 128 & & & & & & 8 & & & & & & & & & 1 & & \end{array}$$

- Signed integer two's complement representation
 - Range for n-bits: $[-2^{n-1}, 2^{n-1} - 1]$

Example with 8-bit (torch.int8): $[-128, 127]$



$$\begin{array}{cccccccc} -2^7 & + & 0 & + & 0 & + & 0 & + & 0 & + & 2^3 & + & 0 & + & 0 & + & 2^0 & = & -119 \\ -128 & & & & & & 8 & & & & & & & & & 1 & & \end{array}$$

Integer – PyTorch – Ranges

- Unsigned integer: $[0, 2^n - 1]$

```
# torch.uint8
torch.iinfo(torch.uint8)
```

two "i"s 🙄

```
iinfo(min=0, max=255, dtype=uint8)
```



Quantization

Floating Point

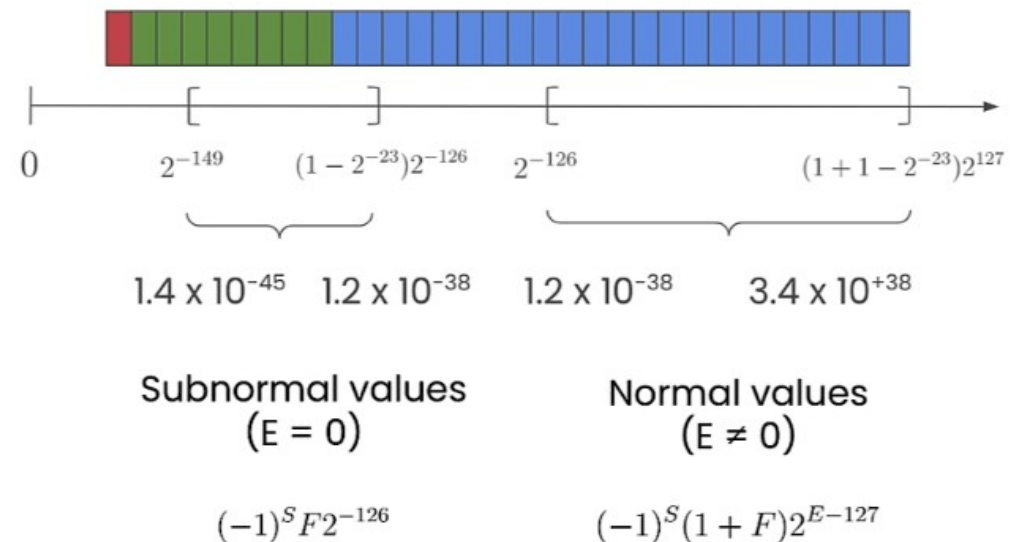
3 components in floating point:

- **Sign**: positive/negative (always 1 bit)
- **Exponent** (range): impact the representable range of the number
- **Fraction** (precision): impact on the precision of the number

FP32, BF16, FP16, FP8 are floating point format with a specific number of bits for **exponent** and the **fraction**.

FP32

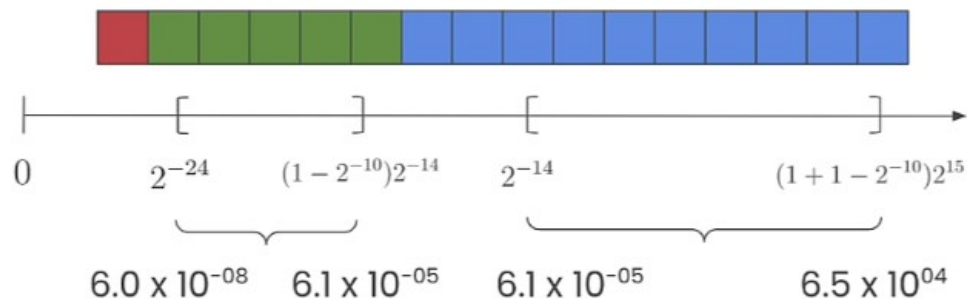
- **Sign**: 1 bit
- **Exponent** (range): 8 bit
- **Fraction** (precision): 23 bit
- **Total**: 32 bit



Quantization

FP16

- **Sign:** 1 bit
- **Exponent** (range): 5 bit
- **Fraction** (precision): 10 bit
- **Total:** 16 bit



Subnormal values
($E = 0$)

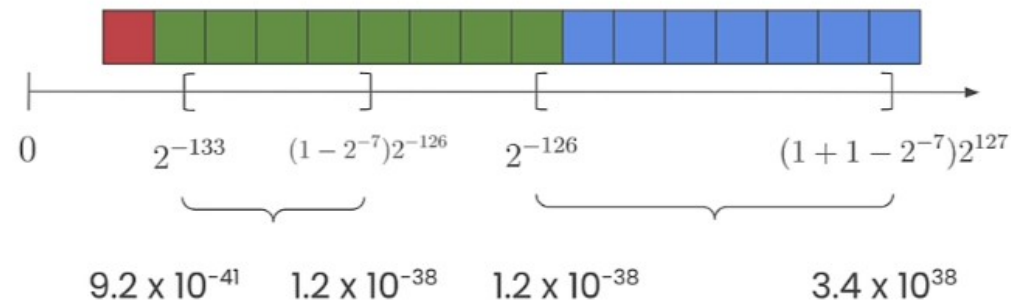
$$(-1)^S F 2^{-14}$$

Normal values
($E \neq 0$)

$$(-1)^S (1 + F) 2^{E-15}$$

BF16

- **Sign:** 1 bit
- **Exponent** (range): 8 bit
- **Fraction** (precision): 7 bit
- **Total:** 16 bit



Subnormal values
($E = 0$)

$$(-1)^S F 2^{-126}$$

Normal values
($E \neq 0$)

$$(-1)^S (1 + F) 2^{E-127}$$

Quantization

Floating Point – PyTorch Downcasting

Comparison

Data Type	Precision	maximum
FP32	best	$\sim 10^{+38}$
FP16	better	$\sim 10^{04}$
BF16	good	$\sim 10^{38}$ 🤗

Advantages:

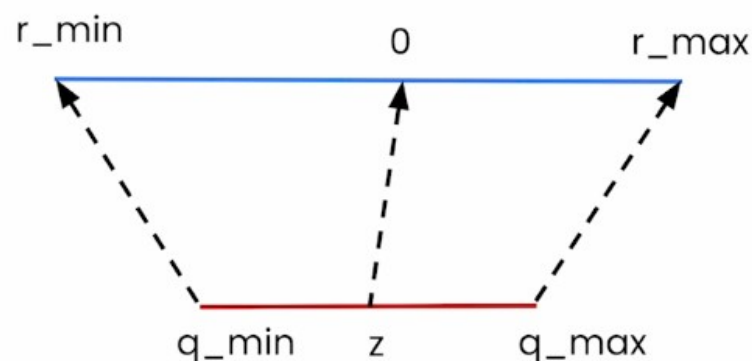
- **Reduced memory footprint.**
 - More efficient use of GPU memory.
 - Enables the training of larger models
 - Enables larger batch sizes
- **Increased compute and speed**
 - Computation using low precision (fp16, bf16) can be faster than fp32 since it require less memory.
 - Depends on the hardware (e.g. Google TPU, NVIDIA A100)

Disadvantages:

- **Less precise** : We are using less memory, hence the computation is less precise.

Quantization

(Optional) Linear Quantization



Simple idea: **linear mapping**

Formula: $r = s(q - z)$

original value
(e.g. in FP32)

quantized value
(e.g. in INT8)

Scale
(e.g. in FP32)

Zero point
(e.g. INT8)

(Optional) scale and zero point

Linear quantization maps the floating point range $[r_{\min}, r_{\max}]$ to the quantized range $[q_{\min}, q_{\max}]$

If we look that the **extreme values**, we should get:

$$\begin{cases} r_{\min} = s(q_{\min} - z) \\ r_{\max} = s(q_{\max} - z) \end{cases}$$

A smaller version of the diagram from the first section, showing the mapping from $q_{\min}$ to $r_{\min}$ and $q_{\max}$ to $r_{\max}$ via the zero point z to 0 .

If we subtract the first equation from the second one, we get the **scale s**:

$$s = (r_{\max} - r_{\min}) / (q_{\max} - q_{\min})$$

For the **zero point z**, we need to round the value since it is a n-bit integer:

$$z = \text{int}(\text{round}(q_{\min} - r_{\min} / s))$$



Hands-On

Notebook 2 - Quantization

BitsAndBytes

bitsandbytes

downloads 18M downloads/month 2M downloads/week 240k

The `bitsandbytes` library is a lightweight Python wrapper around CUDA custom functions, in particular 8-bit optimizers, matrix multiplication (`LLM.int8()`), and 8 & 4-bit quantization functions.

The library includes quantization primitives for 8-bit & 4-bit operations, through `bitsandbytes.nn.Linear8bitLt` and `bitsandbytes.nn.Linear4bit` and 8-bit optimizers through `bitsandbytes.optim` module.

There are ongoing efforts to support further hardware backends, i.e. Intel CPU + GPU, AMD GPU, Apple Silicon. Windows support is quite far along and is on its way as well.

```
class transformers.BitsAndBytesConfig
```

<source>

```
( load_in_8bit = False, load_in_4bit = False, llm_int8_threshold = 6.0, llm_int8_skip_modules = None,
  llm_int8_enable_fp32_cpu_offload = False, llm_int8_has_fp16_weight = False, bnb_4bit_compute_dtype =
  None, bnb_4bit_quant_type = 'fp4', bnb_4bit_use_double_quant = False, bnb_4bit_quant_storage = None,
  **kwargs )
```


BitsAndBytes

Quickstart

The basic way to load a model in 4bit is to pass the argument `load_in_4bit=True` when calling the `from_pretrained` method by providing a device map (pass `"auto"` to get a device map that will be automatically inferred).

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("facebook/opt-350m", load_in_4bit=True, device_map=
...

```

That's all you need!

As a general rule, we recommend users to not manually set a device once the model has been loaded with `device_map`. So any device assignment call to the model, or to any model's submodules should be avoided after that line - unless you know what you are doing.

Keep in mind that loading a quantized model will automatically cast other model's submodules into `float16` dtype. You can change this behavior, (if for example you want to have the layer norms in `float32`), by passing `torch_dtype=dtype` to the `from_pretrained` method.

```
import torch
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.bfloat16
)
```

BitsAndBytes

Advanced usage

You can play with different variants of 4bit quantization such as NF4 (normalized float 4 (default)) or pure FP4 quantization. Based on theoretical considerations and empirical results from the paper, we recommend using NF4 quantization for better performance.

Other options include `bnb_4bit_use_double_quant` which uses a second quantization after the first one to save an additional 0.4 bits per parameter. And finally, the compute type. While 4-bit bitsandbytes stores weights in 4-bits, the computation still happens in 16 or 32-bit and here any combination can be chosen (float16, bfloat16, float32 etc).

The matrix multiplication and training will be faster if one uses a 16-bit compute dtype (default torch.float32). One should leverage the recent `BitsAndBytesConfig` from transformers to change these parameters. An example to load a model in 4bit using NF4 quantization below with double quantization with the compute dtype bfloat16 for faster training:

```
from transformers import BitsAndBytesConfig

nf4_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16
)
```

BitsAndBytes

bfloat16 is the ideal `compute_dtype` if your hardware supports it. While the default `compute_dtype`, `float32`, ensures backward compatibility (due to wide-ranging hardware support) and numerical stability, it is large and slows down computations. In contrast, `float16` is smaller and faster but can lead to numerical instabilities. `bfloat16` combines the best aspects of both; it offers the numerical stability of `float32` and the reduced memory footprint and speed of a 16-bit data type. Check if your hardware supports `bfloat16` and configure it using the `bnb_4bit_compute_dtype` parameter in [BitsAndBytesConfig](#)!



Hands-On

Notebook 3 - BitsAndBytes

llama.cpp



Description

The main goal of `llama.cpp` is to enable LLM inference with minimal setup and state-of-the-art performance on a wide variety of hardware - locally and in the cloud.

Release Madness!

Releases 2,185

 **b3531** Latest

2 hours ago

+ 2,184 releases

5 hours ago

 github-actions

 b3529

 2d5dd7b 

Compare

5 hours ago

 github-actions

 b3528

 cdd1889 

Compare

8 hours ago

 github-actions

 b3527

 c21a896 

Compare

b3529

ggml : add epsilon as a parameter for group_norm (#8818)

Signed-off-by: Molly Sophia <mollysophia379@gmail.com>

Assets 20



b3528

convert : add support for XLMRoberta embedding models (#8658)

* add conversion for bge-m3; small fix in unigram tokenizer

* clean up and simplify XLMRoberta conversion

Assets 20



1

1 person reacted

b3527

[CANN]: Fix ggml_backend_cann_buffer_get_tensor (#8871)

* cann: fix ggml_backend_cann_buffer_get_tensor

1. fix data ptr offset
2. enable the acquisition of incomplete tensors

GGUF

GGUF

GGUF is a file format for storing models for inference with GGML and executors based on GGML. GGUF is a binary format that is designed for fast loading and saving of models, and for ease of reading. Models are traditionally developed using PyTorch or another framework, and then converted to GGUF for use in GGML.

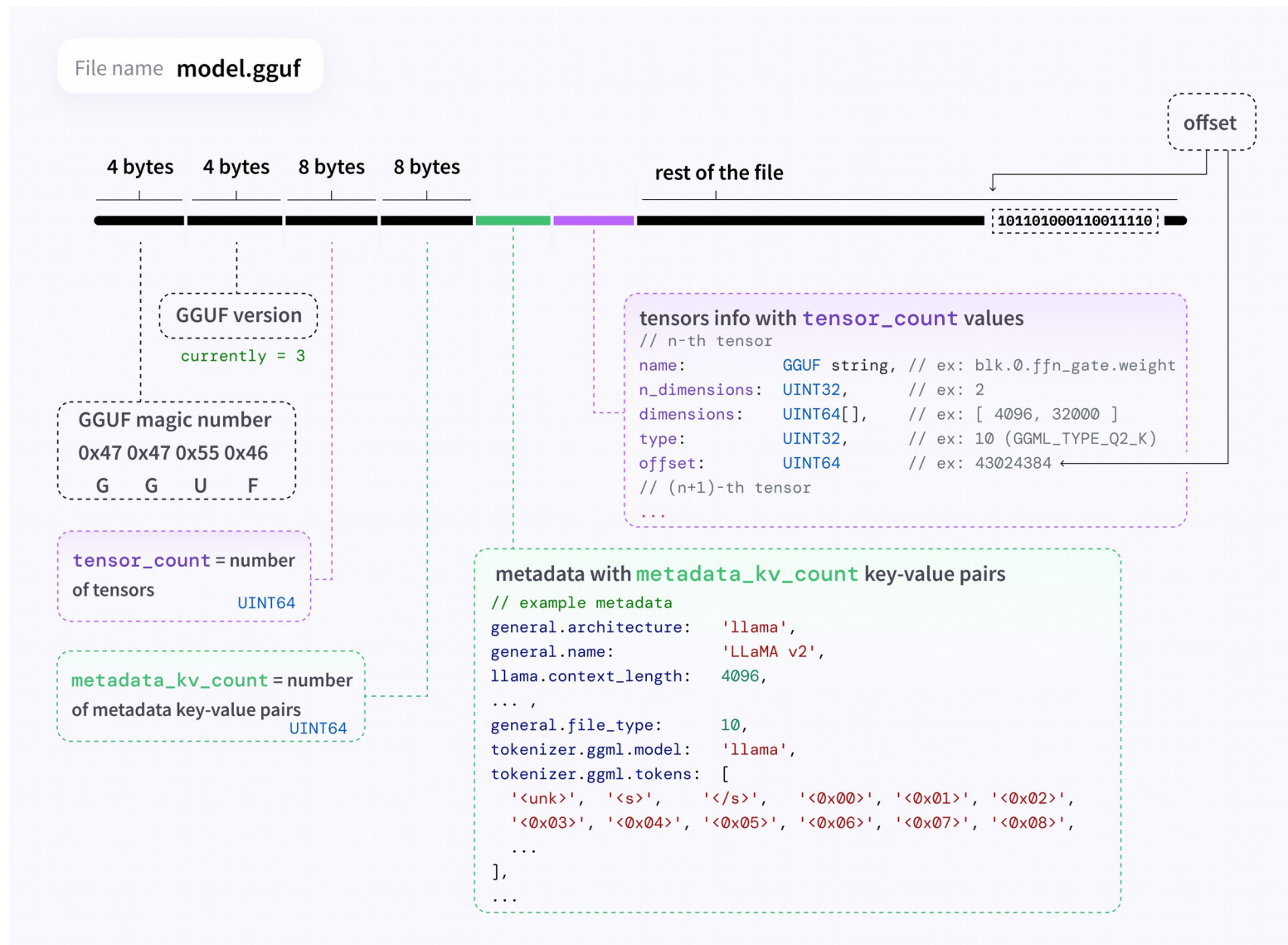
It is a successor file format to GGML, GGMF and GGJT, and is designed to be unambiguous by containing all the information needed to load a model. It is also designed to be extensible, so that new information can be added to models without breaking compatibility.

For more information about the motivation behind GGUF, see [Historical State of Affairs](#).

GGUF

Hugging Face Hub supports all file formats, but has built-in features for [GGUF format](#), a binary format that is optimized for quick loading and saving of models, making it highly efficient for inference purposes. GGUF is designed for use with GGML and other executors. GGUF was developed by [@ggerganov](#) who is also the developer of [llama.cpp](#), a popular C/C++ LLM inference framework. Models initially developed in frameworks like PyTorch can be converted to GGUF format for use with those engines.

GGUF - File Format





Hands-On

Notebook 4 - Converting to GGUF

Notebook 4a - Converting Adapters

Ollama



Get up and running with large language models.

Run [Llama 3](#), [Phi 3](#), [Mistral](#), [Gemma](#), and other models. Customize and create your own.

Download ↓

Available for macOS, Linux,
and Windows (preview)

Ollama - Model Library

Model library

Ollama supports a list of models available on ollama.com/library

Here are some example models that can be downloaded:

Model	Parameters	Size	Download
Llama 3	8B	4.7GB	<code>ollama run llama3</code>
Llama 3	70B	40GB	<code>ollama run llama3:70b</code>
Phi-3	3.8B	2.3GB	<code>ollama run phi3</code>
Mistral	7B	4.1GB	<code>ollama run mistral</code>
Neural Chat	7B	4.1GB	<code>ollama run neural-chat</code>
Starling	7B	4.1GB	<code>ollama run starling-lm</code>
Code Llama	7B	3.8GB	<code>ollama run codellama</code>
Llama 2 Uncensored	7B	3.8GB	<code>ollama run llama2-uncensored</code>
LLaVA	7B	4.5GB	<code>ollama run llava</code>
Gemma	2B	1.4GB	<code>ollama run gemma:2b</code>
Gemma	7B	4.8GB	<code>ollama run gemma:7b</code>
Solar	10.7B	6.1GB	<code>ollama run solar</code>

Note: You should have at least 8 GB of RAM available to run the 7B models, 16 GB to run the 13B models, and 32 GB to run the 33B models.

Ollama - Model Details

phi

Phi-2: a 2.7B language model by Microsoft Research that demonstrates outstanding reasoning and language understanding capabilities.

3B

↓ 101.1K Pulls ⌚ Updated 3 months ago

2.7b



🔖 18 Tags

ollama run phi



Updated 4 months ago		e2fd6321a5fe · 1.6GB
model	arch phi2 · parameters 2.8B · quantization Q4_0	1.6GB
params	{"stop":["User:", "Assistant:", "System:"]}	42B
template	{{ if .System }}System: {{ .System }}{{ end }} User: {{ .P...	77B
system	A chat between a curious user and an artificial intelligen...	132B
license	MIT License Permission is hereby granted, free of charge, ...	1.0kB

Ollama - Custom Model

Customize a model

Import from GGUF

Ollama supports importing GGUF models in the Modelfile:

1. Create a file named `Modelfile`, with a `FROM` instruction with the local filepath to the model you want to import.

```
FROM ./vicuna-33b.Q4_0.gguf
```



2. Create the model in Ollama

```
ollama create example -f Modelfile
```



3. Run the model

```
ollama run example
```



Ollama - Model File

```
> ollama show --modelfile llama3
# Modelfile generated by "ollama show"
# To build a new Modelfile based on this one, replace the FROM line with:
# FROM llama3:latest
FROM /Users/pdevine/.ollama/models/blobs/sha256-00e1317cbf74d901080d7100f57580ba8dd8de57203072dc6f668324ba545f29
TEMPLATE """{{ if .System }}<|start_header_id|>system<|end_header_id|>

{{ .System }}<|eot_id|>{{ end }}{{ if .Prompt }}<|start_header_id|>user<|end_header_id|>

{{ .Prompt }}<|eot_id|>{{ end }}<|start_header_id|>assistant<|end_header_id|>

{{ .Response }}<|eot_id|>"""
PARAMETER stop "<|start_header_id|>"
PARAMETER stop "<|end_header_id|>"
PARAMETER stop "<|eot_id|>"
PARAMETER stop "<|reserved_special_token|>"
```



Ollama

REST API

Ollama has a REST API for running and managing models.

Generate a response

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3",
  "prompt": "Why is the sky blue?"
}'
```



Chat with a model

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3",
  "messages": [
    { "role": "user", "content": "why is the sky blue?" }
  ]
}'
```





Hands-On

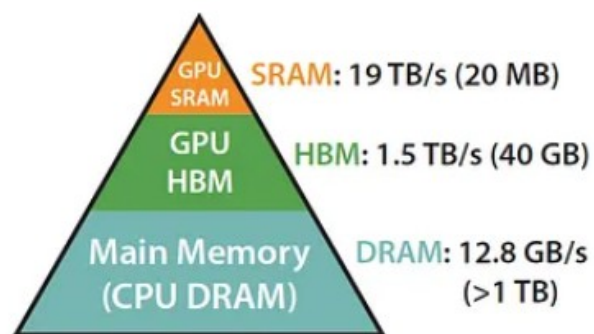
Notebook 5 - Running Ollama



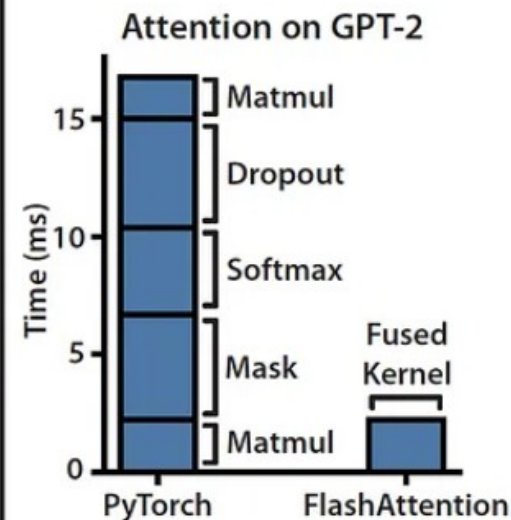
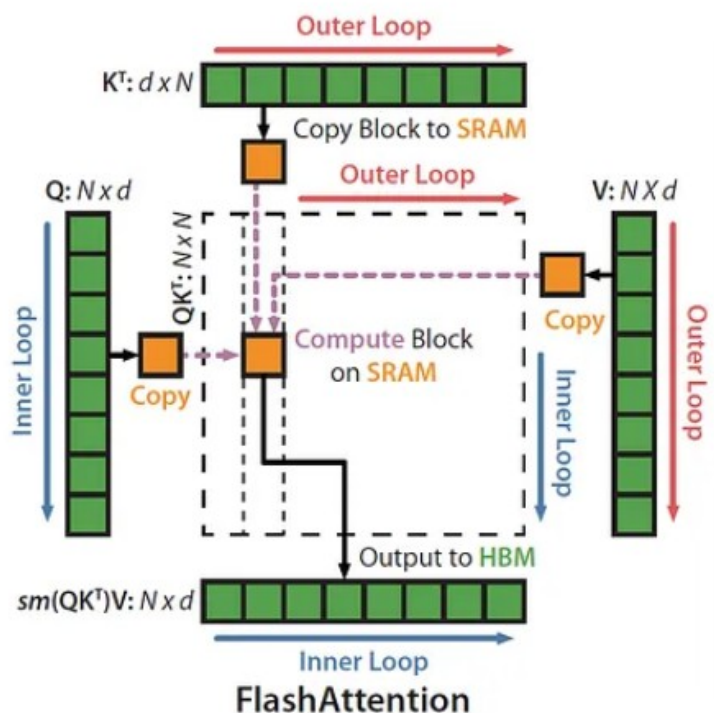
Exercise

Exercise – Quantizing and Serving a Model

BONUS: Flash Attention

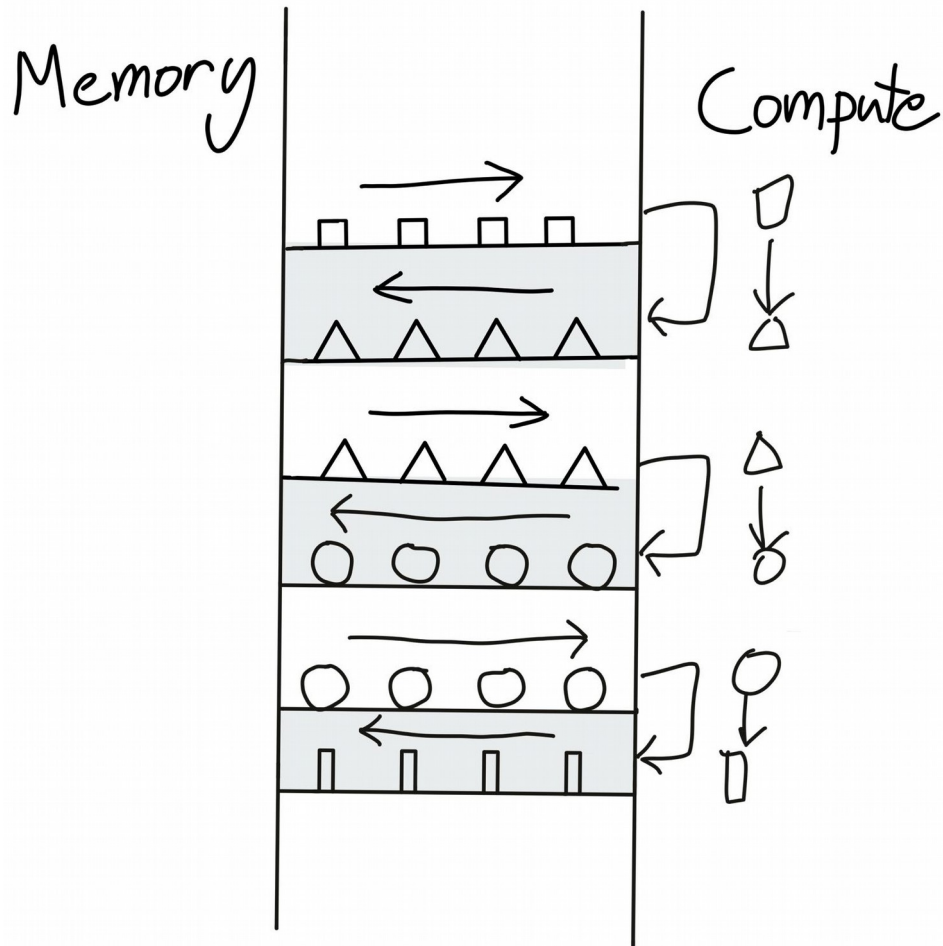


Memory Hierarchy with Bandwidth & Memory Size

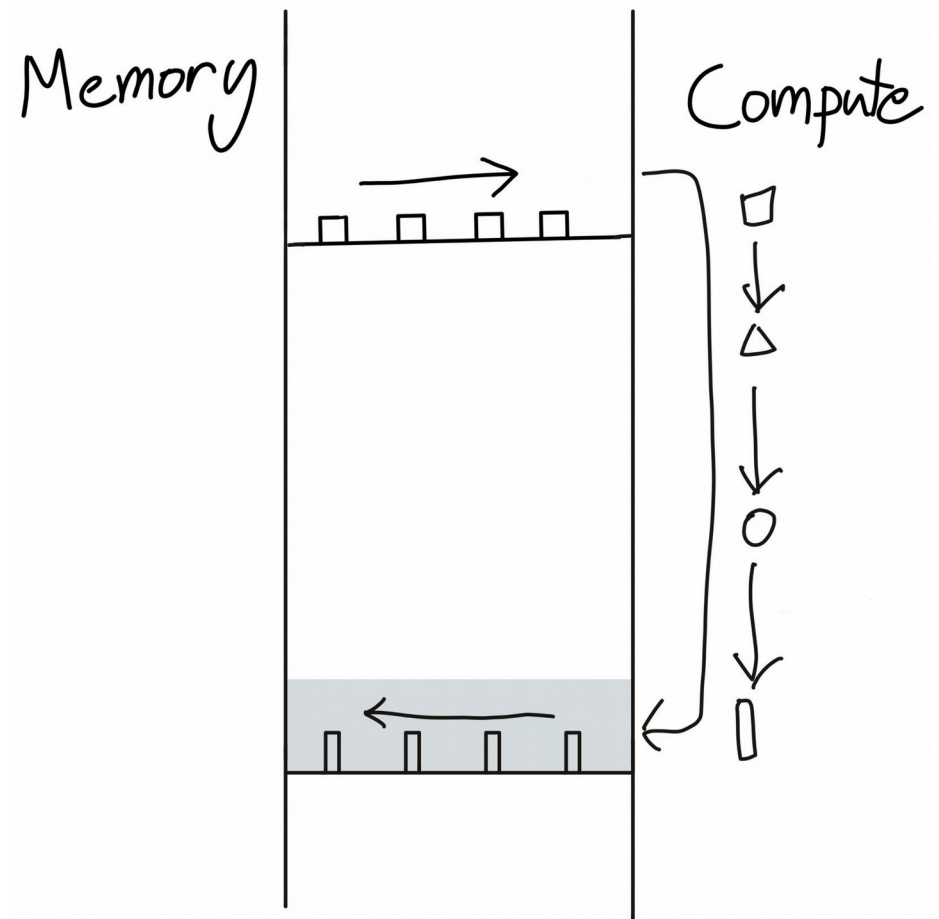


Flash Attention

BEFORE



AFTER



Flash Attention

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

Tiling Softmax

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$m(x) := \max_i x_i, \quad f(x) := [e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}]$$

$$\ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}$$

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = [x^{(1)} \ x^{(2)}] \in \mathbb{R}^{2B}$ as:

$$m(x) = m([x^{(1)} \ x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \begin{bmatrix} e^{m(x^{(1)}) - m(x)} f(x^{(1)}) & e^{m(x^{(2)}) - m(x)} f(x^{(2)}) \end{bmatrix},$$

$$\ell(x) = \ell([x^{(1)} \ x^{(2)}]) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$



Hands-On

BONUS - Flash Attention



The End

THANK YOU!